

PRACTICAL—2

Name:- Sahil Rahane

PRN:- 202501050009

2.1.1. List operations

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- **Add:** Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display:** Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit:** Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Code:

```
l1 = []

while True:

    print("1. Add")

    print("2. Remove")

    print("3. Display")

    print("4. Quit")

    n = int(input("Enter choice: "))

    if n == 1:

        add = int(input("Integer: "))

        l1.append(add)

        print(f'List after adding: {l1}')

    elif n == 2:

        if len(l1) == 0:
```

```

print("List is empty")

elif len(l1) !=0:

remove=int(input("Integer: "))

if remove not in l1 :

print("Element not found")

else:

l1.remove(remove)

print(f"List after removing: {l1}")

elif n==3:

if len(l1) ==0:

print("List is empty")

else:

print(l1)

elif n==4:

break

else:

print("Invalid choice")

```

The screenshot displays a Jupyter Notebook environment. On the left, a sidebar shows the course structure for 'Essentials of Data Science Laboratory - 2304102L - 2026'. The main area is titled '2.1.1. List operations' and contains a problem statement and sample test cases. The problem statement asks for a Python program with a menu-driven interface for managing a list of integers. The sample test cases include adding, removing, displaying, and quitting the list. On the right, the 'listOps.py' file contains the implementation code, which uses a while loop and conditional statements to handle the menu options.

2.1.1. List operations

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:

- Add:** Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- Display:** Displays the current list of integers. If the list is empty, display "List is empty".
- Quit:** Exits the program.

The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Sample Test Cases

```

1 # write the code..
2 l1=[]
3 while True:
4     print("1. Add")
5     print("2. Remove")
6     print("3. Display")
7     print("4. Quit")
8     n = int(input("Enter choice: "))
9     if n == 1:
10        add = int(input("Integer: "))
11        l1.append(add)
12        print(f"List after adding: {l1}")
13    elif n == 2:
14        if len(l1) == 0:
15            print("List is empty")
16        elif len(l1) != 0:
17            remove=int(input("Integer: "))
18            if remove not in l1 :
19                print("Element not found")
20        else:
21            l1.remove(remove)
22            print(f"List after removing: {l1}")
23    elif n == 3:
24        if len(l1) == 0:
25            print("List is empty")

```

2.1.2. Dictionary Operations

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

- All operations must be performed using dictionary methods.
- Perform deletion only if possible; leave the dictionary unchanged.
- Refer to the visible test cases for better understanding and strictly match with the input/outputs.

Code:

Initial dictionary with 10 predefined records

```
student = {  
    1: "Amit",  
    2: "Riya",  
    3: "Kiran",  
    4: "Neha",  
    5: "Arjun",  
    6: "Pooja",  
    7: "Rahul",  
    8: "Sneha",  
    9: "Vikram",  
    10: "Anjali"  
}
```

write your code here...

```
print("Original Dictionary:",student)
```

```
key1=int(input())
```

```
value1=input()
```

```
student.update({key1:value1})
```

```
print("After Insertion:",student)
```

```
key2=int(input())
```

```
value2=input()
```

```
if(key2 in student.keys()):
```

```
student.update({key2:value2})
```

```
print("After Update:",student)
```

```
key3=int(input())
```

```
if(key3 in student.keys()):
```

```
student.pop(key3)
```

```
print("After Deletion:",student)
```

```
print("Traversing Dictionary:")
```

```
for key,value in student.items():
```

```
print(f'{key} : {value}')
```

The screenshot displays a web application interface. On the left, a sidebar shows a course structure for 'Essentials of Data Science Laboratory - 2304102L - 2026'. The main content area is titled '2.1.2. Dictionary Operations' and contains instructions for writing a Python program to perform insertion, update, deletion, and traversal on a dictionary. It lists four operations to be performed: Insertion, Update, Deletion, and Traversal. Below these, it specifies the input and output format. On the right, a code editor shows the implementation of the program. The code defines a dictionary 'student' with 10 predefined records, performs the operations as described, and prints the results at various stages.

```
# Initial dictionary with 10 predefined records
student = {
    1: "Amit",
    2: "Riya",
    3: "Kiran",
    4: "Neha",
    5: "Arjun",
    6: "Pooja",
    7: "Rahul",
    8: "Sneha",
    9: "Vikram",
    10: "Anjali"
}

print("Original Dictionary:",student)

key1=int(input())
value1=input()
student.update({key1:value1})
print("After Insertion:",student)

key2=int(input())
value2=input()
if(key2 in student.keys()):
    student.update({key2:value2})
print("After Update:",student)

key3=int(input())
if(key3 in student.keys()):
    student.pop(key3)
print("After Deletion:",student)

print("Traversing Dictionary:")
for key,value in student.items():
    print(f'{key} : {value}')
```

2.2.1. Linear search Technique

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

Output format:

- If the element is found, print the index. If the element found multiple times, print the starting index
- If the element is not found, print **Not found**.

Sample Test Case:**Input:**

1 2 3 4 3 5 6

3

Output:

2

Code:

```
def linear_search(arr, key):
```

```
    for i in range(len(arr)):
```

```
        if arr[i]==key:
```

```
            return i
```

```
    return -1
```

```
arr = list(map(int, input().split()))
```

```
key = int(input())
```

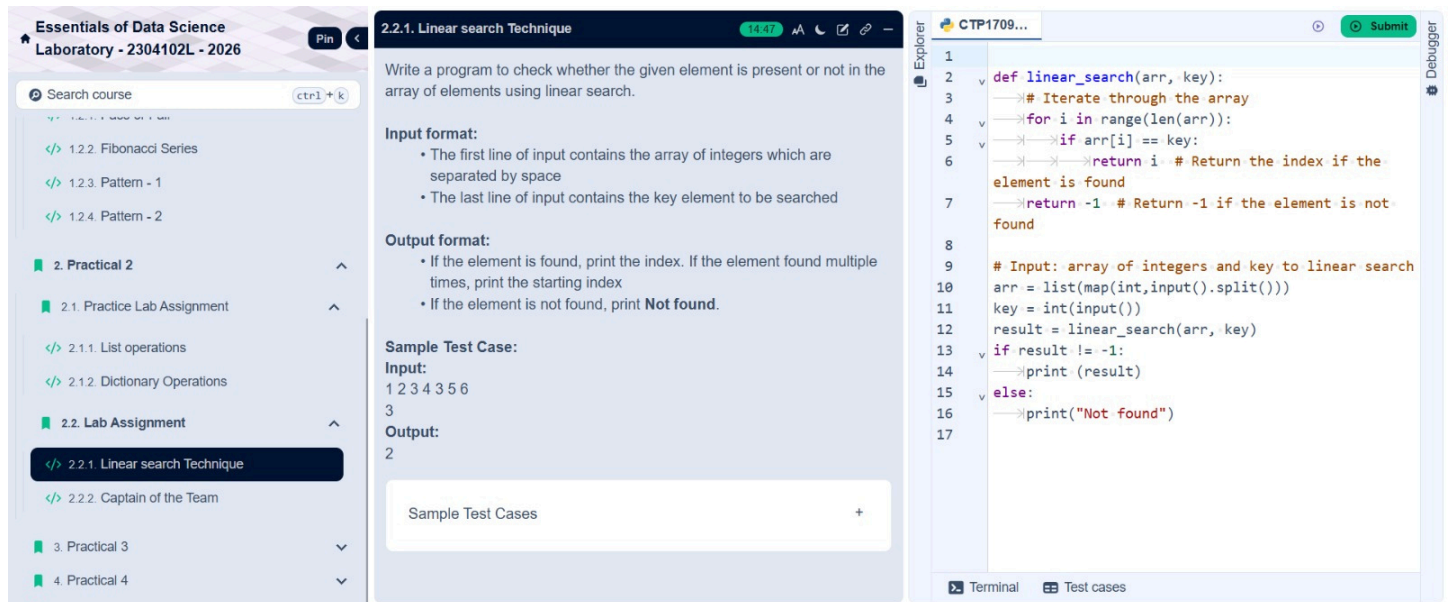
```
result= linear_search(arr,key)
```

```
if result != -1:
```

```
    print(result)
```

```
else:
```

```
    print("Not found")
```



2.2.2. Captain of the Team

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format

The output should be the height (in centimeters) of the tallest player.

Code:

```
heights=list(map(int, input().split()))  
tallest_player = max(heights)  
print(tallest_player)
```

Essentials of Data Science
Laboratory - 2304102L - 2026

Search course

1.2.2. Fibonacci Series

1.2.3. Pattern - 1

1.2.4. Pattern - 2

2. Practical 2

2.1. Practice Lab Assignment

2.1.1. List operations

2.1.2. Dictionary Operations

2.2. Lab Assignment

2.2.1. Linear search Technique

2.2.2. Captain of the Team

3. Practical 3

4. Practical 4

5. Practical 5

2.2.2. Captain of the Team

04:56

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:
The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format
The output should be the height (in centimeters) of the tallest player.

Sample Test Cases

captainof...

Submit

1 heights = list(map(int,input().split()))
2 tallest_player = max(heights)
3 print(tallest_player)

Terminal Test cases

< Prev

Reset

Submit

Next >